KV Cache: Core of Inference Acceleration

□□ [Kwon et al., 2023] [Vaswani et al., 2017]

No KV Cache

Generate token 1:

□□ [The, algorithm, processes, data] □ Q, K, V

4 tokens □ 4x4 attention = 16 ops

Generate token 2:

Recompute [The, algorithm, processes, data, efficiently]

5 tokens □ 5x5 attention = 25 ops □□ Recompute!

Generate token 3:

Recompute again 6 tokens □ 36 ops □□□□

Total complexity: O(n² × m²)

With KV Cache

Prefill Stage:

Compute Q, K, V for [The, algorithm, processes, data]

□ Cache K, V vectors

Generate token 2 (Decode):

Only compute Q, K, V for "efficiently"

Attention with cached K, V □ 5 ops □

Generate token 3 (Decode):

Only compute 1 new token □ attention with cache □

Per-step complexity: O(n+m)

Why cache only K and V, not Q?

Q is only used by the current token to compute attention weights, then discarded. K and V are needed by every future token—"efficiently"'s Q asks "Who should I attend to?", while "The/algorithm/processes/data"'s K/V answer "Here's my info"

KV Cache Memory Usage Comparison Across Models

Model	Hidden Dim	Layers	Seq Length	KV Cache (FP16)	Batch=32
GPT-3 175B	12,288	96	2,048	~9.2 GB	~295 GB
LLaMA-3 8B	4,096	32	8,192	~4.3 GB	~137 GB